

Classroom B - C Lab Worksheet (Single File)

Student Task

Build **one single C file** named `week5_lab_b.c`.

Rules:

1. Use only one `#include <stdio.h>`.
2. Put each question in a separate function.
3. Write only one `main()` and call all question functions inside `main()` in order.
4. Questions 1-5 are normal coding tasks.
5. Questions 6-10 are intentionally broken; fix them first.

Total: 100 points

Each question is worth **10 points**.

Suggested `main()`:

```
int main(void) {
    example1_shadow_copy();
    example2_change_or_protect();
    example3_search_report();
    example4_first_occurrence_finder();
    example5_direct_unique_printer();
    example6_broken_array_snapshot();
    example7_broken_reverse_echo();
    example8_broken_min_max_messenger();
    example9_broken_reference_surprise();
    example10_broken_frequency_counter();
    return 0;
}
```

Question 1 - Shadow Copy [10 points]

Write a function named `array_copy` and test it.

Requirements:

1. Use the source array `int A[5] = {1, 3, 5, 2, 7};`
2. Create the destination array `int B[5];`
3. Copy all elements of `A` into `B`.
4. Print both arrays after the copy.

Rule:

- Use a `while` loop inside `array_copy`.

Question 2 - Change or Protect? [10 points]

Compare two array-processing styles.

Requirements:

1. Write `foo_v1(int A[], int size)` so that it squares each element of the original array and returns the sum of the squared values.
2. Write `foo_v2(int K[], int size)` so that it returns the same sum but keeps the original array unchanged.
3. Use `array_copy` inside `foo_v2`.
4. Test both versions on:
 - `int A[5] = {1, 3, 5, 2, 7};`
 - `int B[5] = {1, 3, 5, 2, 7};`
5. Print the arrays before and after each function call.

Goal:

- Show the difference between modifying the original array and working on a copied array.
-

Question 3 - Search Report [10 points]

Write `search_in_array` and `index_of_last`, then build a small report.

Requirements:

1. Use `#define SIZE 5`
2. Use `int A[SIZE] = {15, 5, 15, 5, -5};`
3. Use `int n = 5;`
4. Print:
 - whether `n` exists in the full array
 - whether `n` exists in the first half
 - whether `n` exists in the second half
 - the last index of `n`

Rule:

- The first-half and second-half checks must use the same pointer style shown in class.
-

Question 4 - First Occurrence Finder [10 points]

Write a function named `index_of_first`.

Requirements:

1. `index_of_first(int A[], int size, int n)` should return the first index of `n`.
2. If `n` is not in the array, return `-1`.
3. Test it on:
 - `int A[5] = {15, 5, 15, 5, 5};`

- `n = 5`
- `n = 99`

Goal:

- This is the natural partner of `index_of_last`.

Question 5 - Direct Unique Printer [10 points]

Print each value only once, in the order of first appearance, **without building a second array**.

Requirements:

1. Use `int A[10] = {50, 0, 8, 50, 5, 50, 50, 8, 5, 57};`
2. Use nested loops.
3. Print a value only if it has not appeared earlier in the array.

Expected output:

```
50 0 8 5 57
```

Question 6 - Broken Code: Array Snapshot [10 points]

The code below is supposed to print the sizes and contents of three arrays. It has several mistakes.

```
void example6_broken_array_snapshot(void) {
    int A[5] = {5, 10, -5};
    int B[] = {5, 10, -5, 4};
    int C[5] = {0};

    printf("# of members of A: %d\n", sizeof(A));
    printf("# of members of B: %d\n", sizeof(B));

    print_int_array(A, sizeof(A) / sizeof(A[0]) - 1);
    print_int_array(B, sizeof(B) / sizeof(B[0]) + 1);
    print_int_array(C, 4);
}
```

Fix it so the reported member counts and printed lengths are correct.

Question 7 - Broken Code: Reverse Echo [10 points]

The function below is supposed to copy an array into another array in reverse order. It has multiple bugs.

```
void array_copy_reverse(int dest[], const int src[], int size) {
    int i, j;
    for(i = 0, j = size - 1; i <= size; i++, j++) {
        dest[i] = src[j];
    }
}
```

Fix it and test it with:

```
int A[5] = {1, 3, 5, 2, 7};
int B[5];
```

Question 8 - Broken Code: Min-Max Messenger [10 points]

The function below is supposed to store the minimum in `Results[0]` and the maximum in `Results[1]`. It has logical bugs.

```
void min_max_of_array(int A[], int size, int Results[]) {
    int i;

    if(A[0] < A[1]) {
        Results[0] = A[1];
        Results[1] = A[0];
    } else {
        Results[0] = A[1];
        Results[1] = A[0];
    }

    i = 2;
    while(i <= size) {
        if(A[i] > Results[1]) Results[0] = A[i];
        else if(A[i] < Results[0]) Results[1] = A[i];
        i++;
    }
}
```

Fix it and test it with:

```
int A[5] = {1, -3, 5, 2, -7};
```

Question 9 - Broken Code: Reference Surprise [10 points]

The function below is supposed to behave exactly like the class example, but it does not.

```
double foo(double m[], int size) {  
    m[1] = 10.0;  
    return m[0] + m[size];  
}
```

Fix it and test it with a `double` array of size 8 whose last element is `4.5`.

The final result should show that the first element changes and the returned sum uses the last valid index.

Question 10 - Broken Code: Frequency Counter [10 points]

The function below should count how many times `n` appears in the array. It has multiple bugs.

```
int count_of_n_normal(int A[], int size, int n) {  
    int count;  
    for(int i = 0; i < size; i++) {  
        if(A[i] = n) count++;  
    }  
    return count;  
}
```

Fix it and test it with:

```
int A[5] = {15, 5, 15, 5, 5};  
int n = 5;
```